

Experiment 2: Production Rules and the Match Phase

Aim

To represent domain knowledge as IF–THEN production rules, hold facts in a working memory, and compute the conflict set of applicable rules.

Theory

A production system has three parts: a rule base of IF–THEN productions, a working memory (WM) of currently believed facts, and an inference engine that runs the recognise–act cycle: match all rules against WM, select one rule from the resulting conflict set, act by executing its conclusion, and repeat.

This experiment isolates the match phase. A rule is applicable when every condition in its IF part is present in WM and its conclusion is not already there (refractoriness — preventing a rule from firing repeatedly on the same data). The set of applicable rules is the conflict set; choosing among them is the subject of Experiment 5.

Algorithm

1. Encode each rule as a record with an identifier, a list of conditions and a conclusion.
2. Initialise working memory as a set of facts.
3. For each rule, test whether all its conditions are members of WM and its conclusion is absent.
4. Collect and display the conflict set.

```

lab2 > main.py > ...
1 # Each rule has:
2 # id -> Rule identifier
3 # conditions -> IF part
4 # conclusion -> THEN part
5
6 # The Conflict Set is simply the list of rules whose conditions are satisfied and whose conclusion is not already present in W
7
8 rules = [
9     {
10         "id": "R1",
11         "conditions": ["has_fever", "has_cough"],
12         "conclusion": "flu"
13     },
14     {
15         "id": "R2",
16         "conditions": ["flu", "body_pain"],
17         "conclusion": "visit_doctor"
18     },
19     {
20         "id": "R3",
21         "conditions": ["has_headache"],
22         "conclusion": "take_rest"
23     },
24     {
25         "id": "R4",
26         "conditions": ["flu"],
27         "conclusion": "take_rest"
28     }
29 ]
30
31 # Working Memory (Current Facts)
32
33 working_memory = {
34     "has_fever",
35     "has_cough",
36     "body_pain"

```

```

lab2 > main.py > ...
37 }
38
39 print("Working Memory:")
40 print(working_memory)
41
42 # Match Phase
43
44 conflict_set = []
45
46 for rule in rules:
47     # Check if all conditions are satisfied for the rule to be applicable
48     conditions_met = all(
49         condition in working_memory
50         for condition in rule["conditions"]
51     )
52
53     # Do not fire if conclusion already exists
54     conclusion_absent = rule["conclusion"] not in working_memory
55
56     if conditions_met and conclusion_absent:
57         conflict_set.append(rule)
58
59 print("\nApplicable Rules (Conflict Set):")
60
61 if conflict_set:
62     for rule in conflict_set:
63         print(f"{rule['id']} : IF {rule['conditions']} THEN {rule['conclusion']}")
64 else:
65     print("No applicable rules.")
66

```

Output:

```
[Running] python -u "d:\IMA lab\lab2\main.py"
Working Memory:
{'has_cough', 'body_pain', 'has_fever'}

Applicable Rules (Conflict Set):
R1 : IF ['has_fever', 'has_cough'] THEN flu
```

Exercises

1. Add facts to WM one at a time and record how the conflict set grows or shrinks.
2. Extend the matcher to support negated conditions of the form 'NOT fact'.
3. Measure (with `time.perf_counter`) how match time grows as you scale to 1,000 auto-generated rules, and relate the result to why the RETE algorithm exists.

```
lab2 > exercises.py > ...
1 import time
2 import random
3
4 # Rule Base
5 rules = [
6     {
7         "id": "R1",
8         "conditions": ["has_fever", "has_cough"],
9         "conclusion": "flu"
10    },
11    {
12        "id": "R2",
13        "conditions": ["flu", "body_pain"],
14        "conclusion": "visit_doctor"
15    },
16    {
17        "id": "R3",
18        "conditions": ["has_headache"],
19        "conclusion": "take_rest"
20    },
21    {
22        "id": "R4",
23        "conditions": ["flu"],
24        "conclusion": "take_rest"
25    },
26    {
27        "id": "R5",
28        "conditions": ["NOT allergy"],
29        "conclusion": "safe_to_take_medicine"
30    }
31 ]
32
33 # Function to find Conflict Set
34 def match_rules(rules, working_memory):
35
36     conflict_set = []
37
```

```

lab2 > exercises.py > ...
34 def match_rules(rules, working_memory):
35
36     for rule in rules:
37
38         conditions_met = True
39
40         for condition in rule["conditions"]:
41
42             # Exercise 2: Handle negated conditions
43             if condition.startswith("NOT "):
44                 fact = condition[4:]
45                 if fact in working_memory:
46                     conditions_met = False
47                     break
48             else:
49                 if condition not in working_memory:
50                     conditions_met = False
51                     break
52
53             # Refractoriness
54             conclusion_absent = rule["conclusion"] not in working_memory
55
56             if conditions_met and conclusion_absent:
57                 conflict_set.append(rule)
58
59     return conflict_set
60
61 # Exercise 1
62 # Add facts one by one and observe Conflict Set
63
64 print("EXERCISE 1 : Growing Working Memory")
65
66 working_memory = set()
67
68 facts_to_add = [
69     "has_fever",
70     "has_cough"
71 ]

```

```

lab2 > exercises.py > ...
72     "body_pain",
73     "flu",
74     "has_headache"
75 ]
76
77 for fact in facts_to_add:
78
79     working_memory.add(fact)
80
81     print("\nAdded Fact ->", fact)
82     print("Working Memory:", working_memory)
83
84     conflict_set = match_rules(rules, working_memory)
85
86     print("Conflict Set:")
87
88     if conflict_set:
89         for rule in conflict_set:
90             print(f" {rule['id']} -> {rule['conclusion']}")
91     else:
92         print(" None")
93
94 # Exercise 2
95 # Negated Conditions
96
97 print("EXERCISE 2 : Negated Conditions")
98
99 working_memory = {
100     "has_fever",
101     "has_cough"
102 }
103
104 print("Working Memory:", working_memory)
105
106 conflict_set = match_rules(rules, working_memory)
107
108 print("Conflict Set:")

```

```

lab2 > exercises.py > ...
112 for rule in conflict_set:
113     print(rule["id"], "->", rule["conclusion"])
114
115 print("\nAdding allergy...")
116
117 working_memory.add("allergy")
118
119 conflict_set = match_rules(rules, working_memory)
120
121 print("Working Memory:", working_memory)
122
123 print("Conflict Set:")
124
125 for rule in conflict_set:
126     print(rule["id"], "->", rule["conclusion"])
127
128 # Exercise 3
129 # Measure Matching Time for 1000 Rules
130
131 print("\nEXERCISE 3 : Performance Test\n")
132
133 large_rules = []
134
135 possible_facts = [f"fact{i}" for i in range(50)]
136
137 for i in range(1000):
138     conditions = random.sample(possible_facts, 3)
139
140     large_rules.append(
141         {
142             "id": f"R{i}",
143             "conditions": conditions,
144             "conclusion": f"new_fact{i}"
145         }
146     )
147
148

```

Output:

```

D:\IMA lab>python lab2/exercises.py
EXERCISE 1 : Growing Working Memory

Added Fact -> has_fever
Working Memory: {'has_fever'}
Conflict Set:
R5 -> safe_to_take_medicine

Added Fact -> has_cough
Working Memory: {'has_fever', 'has_cough'}
Conflict Set:
R1 -> flu
R5 -> safe_to_take_medicine

Added Fact -> body_pain
Working Memory: {'has_fever', 'has_cough', 'body_pain'}
Conflict Set:
R1 -> flu
R5 -> safe_to_take_medicine

Added Fact -> flu
Working Memory: {'has_fever', 'has_cough', 'body_pain', 'flu'}
Conflict Set:
R2 -> visit_doctor
R4 -> take_rest
R5 -> safe_to_take_medicine

Added Fact -> has_headache
Working Memory: {'flu', 'has_headache', 'body_pain', 'has_cough', 'has_fever'}
Conflict Set:
R2 -> visit_doctor
R3 -> take_rest
R4 -> take_rest
R5 -> safe_to_take_medicine
EXERCISE 2 : Negated Conditions
Working Memory: {'has_fever', 'has_cough'}
Conflict Set:
R1 -> flu
R5 -> safe_to_take_medicine

```

```
Adding allergy...  
Working Memory: {'has_fever', 'has_cough', 'allergy'}  
Conflict Set:  
R1 -> flu
```

EXERCISE 3 : Performance Test

```
Total Rules: 1000  
Applicable Rules: 193  
Matching Time: 0.20169999334029853 ms
```

```
Total Rules: 10000  
Applicable Rules: 2067  
Matching Time: 1.53609999968834221 ms
```

Viva Questions

1. Describe the recognise–act cycle.

The recognise–act cycle is the process used by a production system in which the inference engine first matches all rules with the facts in the working memory to form the conflict set, then selects one applicable rule, executes its conclusion, updates the working memory, and repeats the cycle until no more rules can be applied.

2. What is refractoriness and why is it needed?

Refractoriness is the condition that prevents a rule from firing repeatedly on the same data by ensuring that its conclusion is not already present in the working memory. It is needed to avoid redundant rule execution and infinite loops during inference.

3. What problem does the RETE matching algorithm solve?

The RETE matching algorithm solves the problem of inefficient rule matching in large production systems by avoiding repeated comparisons between rules and facts. It stores intermediate matching results so that only changes in the working memory need to be processed, making matching much faster as the number of rules increases.